Timing Closure in Deep Submicron Designs

Nancy D. MacDonald
Principal IC Design Engineer, Enterprise Networking Group
Broadcom Corporation, Irvine, CA, USA

Notice of Copyright

ARTICLE: Timing Closure

Timing Closure in Deep Submicron Designs

Nancy D. MacDonald
Principal IC Design Engineer, Enterprise Networking Group
Broadcom Corporation, Irvine, CA, USA

I. INTRODUCTION

Final timing closure in complex, hierarchical, deep submicron designs is an arduous task. While EDA tools do a very good optimization job, in general, they often leave several hundred setup and hold violations on the table. This is not the fault of the tools: they do what they can while maintaining competitive turnaround times, and design teams are always pushing the limits of tool capabilities and design complexity. Such is the nature of chip design in a competitive world.

For hierarchical designs, timing closure applies to both individual blocks (hard partitions) as well as the top-level. In this paper, we look at timing closure from the top-level perspective, assuming all lower-level blocks are closed or nearly closed. In fact, even the “closed” blocks may see a whole new set of violations when merged with the top-level. This is due to (1) inaccuracies in block-level, interface timing constraints or (2) differences in SI timing windows for block-level, interface nets, particularly clocks. In any event, assume that our EDA tool flow has taken us as far as possible toward closing timing so that most of the remaining violations are small, and all very large violations have been analyzed and removed. In addition, any failures caused by incorrect or incomplete timing constraints have been addressed. What should a designer do to achieve timing closure?

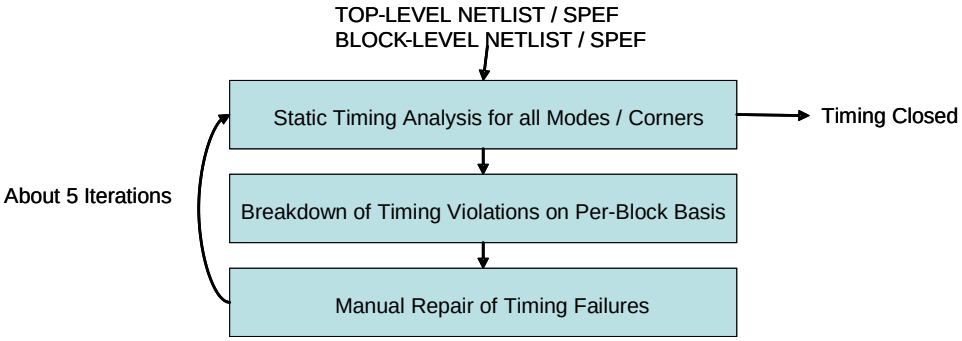
Typically, this last leg of timing closure consists of manual design tweaks and corresponding timing iterations. For large designs, it should take about three weeks to close timing completely (e.g., five manual repair iterations, at three days per iteration, assuming no more than a couple of major tasks per designer). This article starts with high-level principles to follow when finishing timing closure manually, and then gives some specific details.

II. FLOW DIAGRAM

Figure 1 shows a flow diagram for the manual portion of static timing closure. As mentioned earlier, we allow about five top-level timing iterations in the schedule, each taking no more than three to five days. A single iteration consists of the following.

1) Static timing analysis for all modes/corners. We can evaluate QOR in terms of WNS and TNS for setup and hold, max capacitance violations, large transition time violations, and noise violations.
2) Breakdown of timing failures on a per-block basis. Timing violations can be sorted by "failing endpoint". For example if a failing path begins in block A (launch point) and ends in block B (capture point), then the path is placed into the bin for block B. In this way, after "binning" the violations, we assign ownership of all violations to particular block designers. For instance, the engineer responsible for block B owns all violations that end in block B. We assign one engineer to work on violations with top-level endpoints as well. To clarify the concept of "ownership": engineers must fix all of the paths assigned to them during binning. This is true whether or not the failing path originates in their block. It is expected that, in many cases, a designer will need to collaborate with others to resolve a problem (such as the top-level engineer, or the block owner for the source register in a failing path, for example). Such teamwork is essential in a healthy physical design organization.
3) Manual repair of timing failures. The responsible engineers (assigned in step 2) must implement manual repairs on their respective blocks to eliminate timing violations. It is expected that overall timing will improve at the end of this step, although designers may have to accept some bad moves at intermediate points to complete all necessary timing repairs.

At the end of each iteration (three steps described above), we expect to see an improvement in overall top-level timing QOR, although the mix of violations may vary slightly.



Operations Permitted at Each Iteration (in order of preference)

Iteration 1: Vt Swap, Resizing, Buffer Insertion, NDR Changes, Useful Skew
Iteration 2: Vt Swap, Resizing, Buffer Insertion, NDR changes
Iteration 3: Vt Swap, Resizing, Buffer Insertion
Iteration 4: Vt Swap, Resizing
Iteration 5: Vt Swap

Violation Classes Addressed for Each Iteration (in order of priority)
(1) Electrical Rule Violations
(2) Noise Violations
(3) Setup Violations
(4) Hold Violations

Figure 1: Flow Diagram.

In general, we address four main categories of violations during every iteration of the flow described above. The categories are shown below and should be addressed in the following order.

1) Electrical rule violations
2) Noise violations
3) Setup violations
4) Hold violations

We fix electrical rules (such as large max transition and max capacitance) first because we want our design in a "legal" state before taking action on other issues; otherwise, unnecessary design changes (resizing or Vt swap) may occur to address violations that, in reality, stem from the electrical rule failures. Next, we repair noise glitches to ensure a clean noise profile before addressing setup and hold violations. Setup violations are fixed third because the changes needed to repair these failures sometimes have a large ripple effect, whereas hold violations are fixed last because changes to fix these violations are often local and isolated.

To expedite timing closure, we provide some general guidelines as to what types of "fixes" (design changes / transformations) are allowable during each of the five iterations. In general, some changes such as adding useful skew are far more disruptive than others such as changing the Vt class of an instance. We recommend that highly disruptive operations be completed in very early iterations (say 1-2) and prohibited in later ones (say 3-5). The flow diagram in Figure 1 shows what types of operations are permitted at various stages.

In addition, we recommend that, during a particular iteration, operations should be performed in a specific order - from simple and least disruptive to complex and most disruptive. The order of priority is as follows.

1) *Vt swap*. Changing the Vt class of an instance is considered to be a non-disruptive operation. Vt swap does not perturb routing and does not require a new RC extraction after the change. The only drawback is a slight increase in power and the remote possibility of introducing a new noise or even max capacitance violation depending on the characteristics of the new gate.
2) *Resizing*. While not as desirable as Vt swap, resizing is fairly non-invasive. Resizing a gate requires ECO placement. If the resized gate is larger than the original, instances must shift to accommodate the new size. In addition, ECO routing is required to reroute nets for the resized gate and any other affected instances. Note that any perturbation to the routing is undesirable, as new timing violations may be introduced when SI victims / aggressors / virtual attackers change.
3) *Inserting a buffer*. Buffer insertion is somewhat more invasive than either Vt swap or resizing because it causes more variation in ECO placement and routing, particularly in congested designs. Nevertheless, it is sometimes absolutely necessary for the following objectives: (1) load splitting for high-fanout nets or when load cells are spread out, (2) load shielding (example below), (3) repeater insertion for a long net, (4) repeater insertion to remedy a noise failure, or (5) buffer insertion to fix hold time violations. To clarify, an example of load shielding is as follows. Given a high-fanout net with a small number of timing-critical branches, an inserted buffer can serve as the root of a subtree that drives only non-critical branches of the net, thereby reducing the delay for critical branches.
4) *Changing to non-default routing rules (NDR)*. Examples of non-default routing rules include triple spacing or shielding for particular nets. Changing to NDR during the final stages of timing closure is risky, although very effective for eliminating noise glitches or noise-on-delay problems. This fix should not be "over-used" and may not even be possible depending on congestion. The concern is that many nets will be ripped up and rerouted to honor the new NDR.

5) *Adding useful skew*. Useful skew is guaranteed to fix a violating path; however, it inevitably causes other, new violations to pop up. Nevertheless, sometimes there is no other choice. This tactic should be used only as a last resort since the resulting new violations must be fixed manually or even with another pass through the EDA tool before resubmitting the block for top-level integration.

Although we strongly recommend the above guidelines regarding what kind of operations are permitted (and when) during timing closure iterations, we note that exceptions do occur. A few exceptions should always be allowed on a special request basis.

Finally, for all repair passes, designers must be absolutely sure to visualize what is going on in the layout. If problems cannot be solved with a simple Vt swap (or sometimes even when they can), designers must examine the layout to identify the root cause. Is it a long net, a missing non-default rule, poor placement of a few cells, or a detour around a memory or other hard macro? The appropriate course of action often cannot be determined without taking a look at the layout view of the design. Make certain that designers do so regularly.

Below are some specific tips for fixing different types of violations. In addition, we provide loose guidelines for how to address timing failures across various chip modes and corners.

III. GUIDELINES FOR MODES AND CORNERS

Since complex, deep submicron designs have a large number of modes and corners to address, a few simple guidelines can help to reduce complexity.

Rule Number 1: Even if designers are not able to address all violations in only one pass, they should still look at all modes and corners to guide their manual fixes.

This concept is important for several reasons. First, the more modes that are addressed simultaneously the better. We do not want any surprises to appear close to tapeout due to neglected modes/corners. Next, occasionally, the same path cannot meet both setup and hold timing across all corners. Paths such as this must be flagged to the front-end design manager as soon as possible. Certainly, we must take note of any fixes that close timing in one mode but break another.

Rule Number 2: Always identify the dominant corner for setup failures and for hold failures, and fix violations in those corners first.

Although we may look at all modes and corners when evaluating timing results, it is often more efficient to identify one corner to address first when fixing violations. For example, we can usually identify one dominant corner for setup (and another for hold). Fixing violations in the dominant corner clears most of the violations for the others. In other words, if most violations in a design are on memory interfaces, and memories are slowest in the ss/low temperature corner, then ss/low temperature is the dominant corner.

Rule Number 3: Address violations in the main functional mode first.

If block designers cannot address all of their setup/hold violations in a single iteration, they should focus on the main functional mode first, as fixing violations in this mode will likely fix the same

violation in another mode. Also, changes made to a block in main functional mode tend to affect a large number of other blocks, so we should address main functional mode early due to this ripple effect. Then, address the secondary modes and all test modes (in parallel but with lower priority).

IV.FIXING CLOCK NOISE

Clock noise is one of the primary culprits that cause timing closure problems. There are two main issues related to clock noise.

The first problem is that SI timing windows for clock inputs to blocks are frequently not modeled accurately. This is due to inaccurate estimation of clock delays to block inputs and the fact that clock reconvergence pessimism removal doesn't apply to timing windows. Therefore, any OCV derating, clock muxing, etc. will have an impact on the timing windows seen by the blocks. Inaccurate timing windows on clock pins imply that the noise profile for a block, based on the timing constraints, may be different from the one seen when the block is integrated into the top-level of the design. This problem causes failures to appear during top-level timing that were not seen at the block level. Designers can address these violations during manual timing closure, or a methodology change can be adopted to ensure more accurate timing windows.

The second problem is simply noise on clock nets, most frequently caused by one of the following:

1) Weak drivers on long clock nets.
2) Missing or incorrectly applied non-default rules.
3) Clock nets routed on thick metal to cross over macro cells, etc. Thick metal has more sidewall capacitance, so capacitive coupling usually increases (despite increased spacing rules).
4) Inadequate clock max transition or max capacitance rules.

These problems may creep into designs gradually during ECOs and manual or automated timing optimization. Clock noise is a big problem since it translates directly into greater skew between flops.

Consider the example in Figure 2 below. In this case, eliminating the noise on net CLK_A_N would fix a setup violation on REG_A2. CLK_A_N is driven by weak driver BUF_A. To improve the slew rate on net CLK_A_N, we can change BUF_A to a higher-drive cell. This reduces the noise on CLK_A_N. Note that we do not use Vt swap to resolve problems on clock nets because mixing Vt classes on clock trees leads to noise problems and greater on-chip variation.

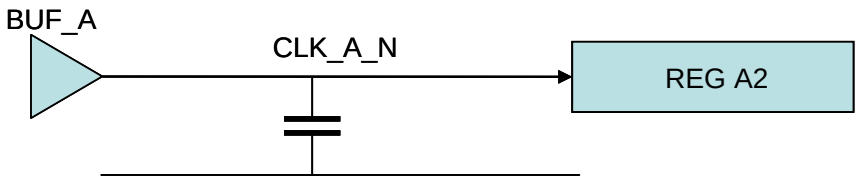


Figure 2: Fixing Clock Noise by Resizing.

In some cases, clock nets are missing non-default rules. In Figure 3, no resizing is possible, as drivers are already maximally sized. The only solution is to apply a non-default rule to CLK_A_N as shown in the figure. In this case, we use triple spacing. Shielding or increasing wire width for net CLK_A_N may also be possible, but would come at a cost in terms of design routability.

In general, note that different NDRs may be applied to different clocks. For example, we might shield an extremely high-frequency system clock while using default rules for a low-frequency test clock. Also, depending on congestion, we might apply an NDR to the upper portion of a large clock tree but allow the leaf nets to be routed with default rules. Designers may employ many different flavors of NDRs, and the best choice is typically design- and process-dependent.

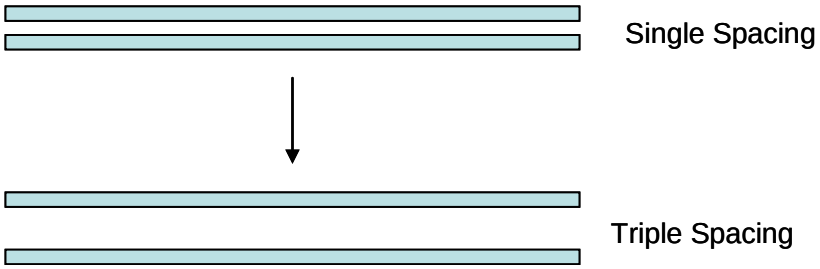


Figure 3: Fixing Clock Noise by NDR.

Noise problems caused by clocks routed in thick metal may also be resolved with non-default rules; however, such problems usually occur in congested areas of the design such as when routing over a macro. It is best to avoid routing clocks in thick metal if at all possible.

The problem of having relaxed clock max transition or max capacitance rules for clocks is systemic and is difficult to resolve with manual tweaks to the clock tree. We just have to live with noise caused by these problems.

V. FIXING NOISE GLITCHES ON SIGNAL NETS

Eliminating noise glitches on signal nets can be done in much the same way as for clock nets. The following operations are permissible.

1) Vt swapping or upsizing the victim net driver.
2) Inserting a repeater on the victim net.
3) Manually moving nets away from the victim.
4) Applying non-default rules to the victim net.

Vt swapping to replace a (victim net's) high Vt driver cell with a standard or low Vt cell is always a good option because it requires no change to the routing at all, assuming that the cell footprints match across all Vt classes. As mentioned previously, routing perturbations during the final stages of timing closure are undesirable because they may easily introduce new violations of virtually any type.

If Vt swap is not possible, since the driving cell of the victim is already low Vt, then upsizing the driver is the second most desirable choice. This will decrease slew rate on the victim net and change the SI timing window.

The third best solution for eliminating noise glitches is repeater insertion. If a victim net is long, the only choice may be to break the net. Repeaters should be inserted approximately halfway between source and sink or inserted in such a way as to break up a high fanout net into relatively "equal" parts. Figure 4 shows a few examples of "good" locations to add repeaters. Note that when fixing noise, one is not trying to balance the capacitive load or speed up any part of the path. The goal is to

eliminate noise by changing timing windows and minimizing coupling. Therefore, we simply try to break the total wire length into equal parts and/or eliminate any long wire segments. Repeater insertion does add delay to a path, but hopefully, the additional delay is cancelled out (and then some) by the reduction in crosstalk delay.

When compared to logic changes, simply manually moving aggressor nets away from a victim net may seem like the best approach. However, it isn't always clear which net(s) to move. When multiple aggressors are involved, it is difficult to assess the effect of per-aggressor and small accumulated-aggressor filtering. We cannot easily/precisely determine how the contributions of each aggressor combine to cause the problem. Furthermore, this approach is very tedious, and if other automated routing changes occur in the neighborhood of the victim, the SI violation may reappear at a later stage in the design cycle. Therefore, we try to avoid this approach because it isn't always as simple as it may appear.

Note that simply applying non-default routing rules to the victim net is also an option, but in a congested area, this may perturb the routing and introduce new violations. Non-default rules for signal nets should be used infrequently and only when there is sure to be minimal fallout.

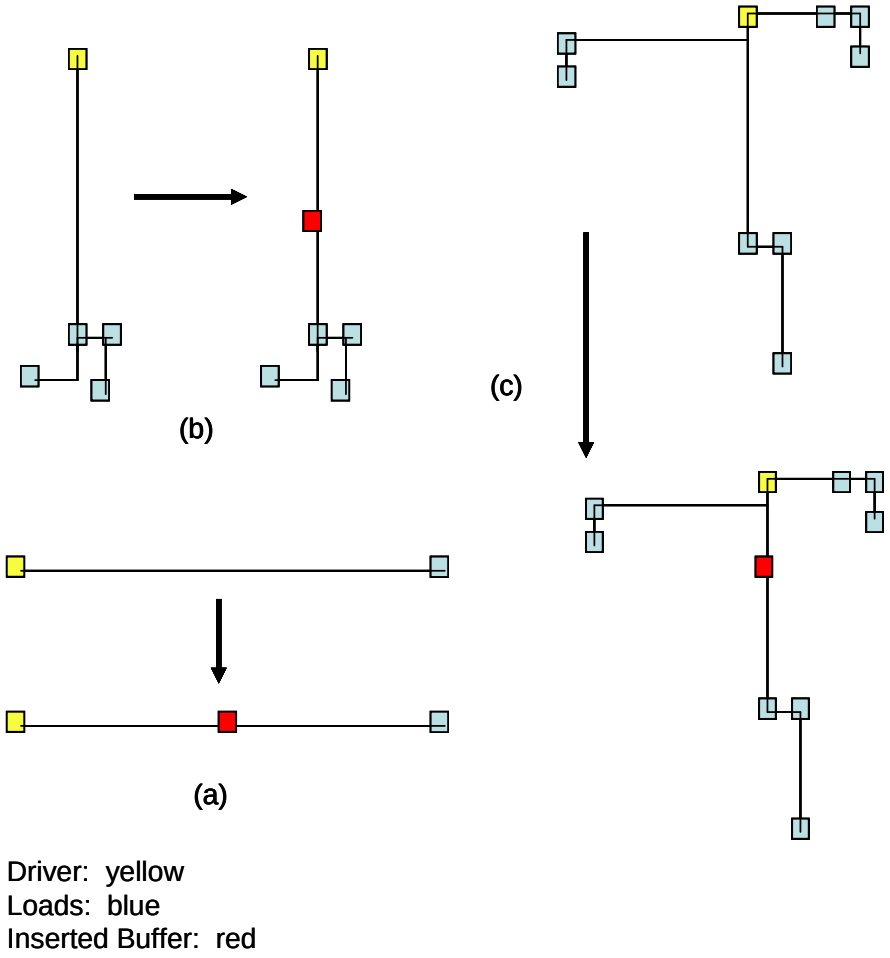


Figure 4: Examples of Buffer Insertion on a Noise Net.

VI.FIXING ELECTRICAL RULE VIOLATIONS

Designers may need to clean up a few electrical rule violations manually. These include: (1) max capacitance violations and (2) large max transition violations.

Max capacitance violations most often occur when library cells use pass transistor logic, as the drive capability of these gates (XOR, MUX, HA carry bit) is quite limited. Figure 5 illustrates buffer insertion to repair a max capacitance violation. In this case, the objective of buffer insertion is to break up the load cells; therefore, we insert three buffers (one for each branch) very close to the driver. Clearly, the number of inserted buffers and the corresponding buffer locations are much different than for noise fixing.

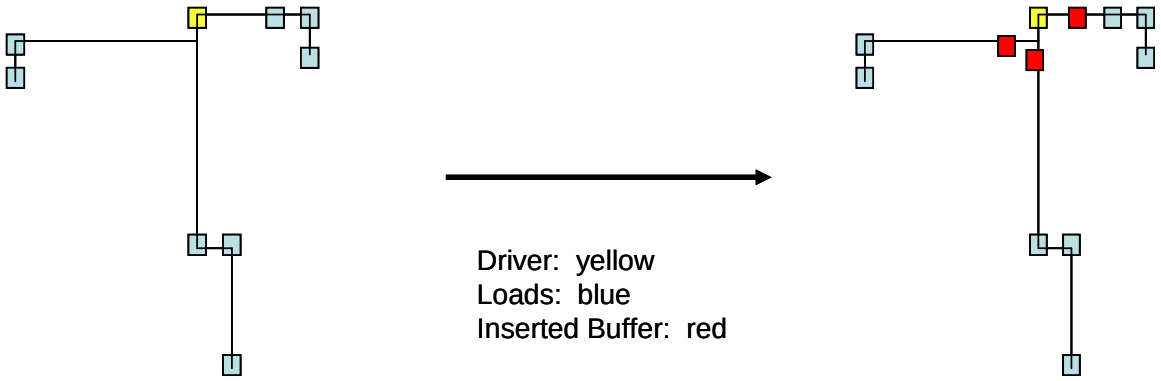


Figure 5: Buffer Insertion to Fix a Max Capacitance Violation.

Other possible resolutions for max capacitance violations include upsizing the driver (yellow) and/or downsizing the loads (blue). Resizing is preferable to buffer insertion since it is less disruptive. In addition to max capacitance failures, we must also fix large max transition violations, typically with repeater insertion. Small slew rate violations can be tolerated or, if possible, repaired by upsizing the driver.

It is important to fix the vast majority of all electrical rule violations in the early repair passes since the necessary buffer insertion may add delay to some paths. Any additional violations that pop up can be fixed in the later repair passes.

VII. FIXING SETUP VIOLATIONS

In order to fix setup violations effectively, we must first report the appropriate data for analysis. Ideally, timing reports will include fanout, capacitive load, net delay, crosstalk delta delay, cell delay and slew rate. A sample path from such a timing report is shown in Figure 7.

To identify the proper fix for a particular violation, we must inspect the report and look for any anomalous data, such as (1) a large crosstalk delay, (2) an unusually large slew rate, or (3) a high fanout, to determine where the problem lies. See Figure 6 below for examples of these. Anomalous

cells and nets are good targets for manual setup fixes, since they have likely not been optimized correctly.

```
     Point                          Fanout    Cap    DTrans    Trans    Delta      Incr       Path
(1) modA/inst1/i3 (SVT_NANDX4)                        0.000    0.074    0.120    0.352 H    3.801r
(2) modA/inst2/o (SVT_BUFX12)                                  0.353             0.907 &    4.008r
(3) modA/net1 (net)                  32    3.148
```

Figure 6: Three Types of Anomalous Timing Data.

Also, designers should browse the timing report to look for bottlenecks. When fixing setup, modify higher-fanout instances, if possible, since they may address many endpoint violations at once.

In general, setup violations fall into several categories.

1) Violations that can only be fixed with useful skew.
2) Violations that can be fixed with buffer insertion.
3) Violations that can be fixed with upsizing.
4) Violations that can be fixed with Vt swap.

Timing reports help us to identify a category for each violation and figure out the appropriate fix.

A. Useful Skew

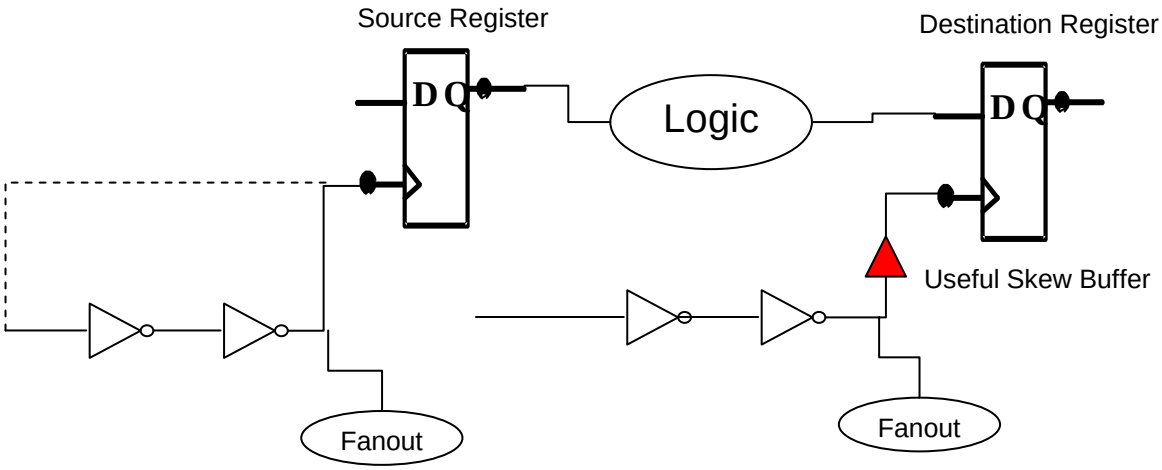
Useful skew causes major upheaval in the design and must only be added during first-iteration repair and only when no other options exist for fixing the failure.

```
Startpoint: data_in
              (input port clocked by clk_vir)
Endpoint: i_mod1/macroA
             (rising edge-triggered flip-flop clocked by clk_ab)
Path Group: INPUT
Path Type: max
Max Data Paths Derating Factor  : 1.09(cell)  1.09(net)
Min Clock Paths Derating Factor : 0.90(cell)  0.90(net)
Max Clock Paths Derating Factor : 1.02(cell)  1.02(net)

Point                       Fanout   Cap    DTrans   Trans    Delta     Incr       Path
-------------------------------------------------------------------------------------------
clock clk_vir (rise edge)                             0.000             0.000      0.000
clock network delay (ideal)                                             1.490      1.490
input external delay                                                    2.000      3.490 r
data_in (in)                                          0.036             0.024 &    3.514 r
data_in (net)                  1    0.008
Ubuf_data_in/i (LVT_BUFX20)
                                            0.000     0.036    0.000    0.000 &    3.514 r
Ubuf_data_in/o (LVT_BUFX20)                           0.054             0.183 &    3.697 r
N100 (net)                     1    0.183
i_mod1/macroA_data_in (macroA_wrapper)                0.000             0.000 H    3.697 r
i_mod1/macroA_data_in (net)
i_mod1/macroAwrap_i2050/i (LVT_BUFX20)      0.000     0.074    0.030    0.104 H    3.801 r
i_mod1/macroAwrap_i2050/o (LVT_BUFX20)                0.053             0.207 &    4.008 r
i_mod1/macroA_33291 (net)
                               1    0.148
i_mod1/macroAwrap_i976/i (LVT_CKBUFX20)
                                            0.000     0.055    0.045    0.065 H    4.073 r
i_mod1/macroAwrap_i976/o (LVT_CKBUFX20)               0.034             0.153 &    4.227 r
i_mod1/macroA_15285 (net)
                               1    0.280
i_mod1/macroAwrap_i2017/i (LVT_BUFX20)      0.000     0.095    0.011    0.164 H    4.391 r
i_mod1/macroAwrap_i2017/o (LVT_BUFX20)                0.060             0.221 &    4.612 r
i_mod1/macroAwrap_32846 (net)
                               1    0.295
i_mod1/macroA/macroA_data_in (macroA_master)
```

```
                                        0.000    0.066    0.000    0.040 &    4.651 r
data arrival time                                                             4.651

clock clk_ab (rise edge)                                                      4.000      4.000
clock source latency                                                          0.250      4.250
clk_ab (in)                                             0.000                 0.000 &    4.250 r
clk_ab (net)                  1    0.003
Ubuf_clk_ab/i (LVT_CKBUFX8)                     0.000   0.000    0.000        0.000 &    4.250 r
Ubuf_clk_ab/o (LVT_CKBUFX8)                             0.024                 0.072 &    4.322 r
clk_ab0 (net)                 2    0.060
clk_ab_go_cts__d184/i (LVT_INVX20)              0.000   0.024    0.000        0.004 &    4.326 r
clk_ab_go_cts__d184/o (LVT_INVX20)                      0.022                 0.046 &    4.372 f
clk_ab_go_cts__d184n (net)    1    0.100
clk_ab_go_cts__d183/i (LVT_INVX20)              0.000   0.022    0.000        0.021 &    4.394 f
clk_ab_go_cts__d183/o (LVT_INVX20)                      0.032                 0.061 &    4.455 r
clk_ab_go_cts__d183n (net)    1    0.117
clk_ab_go_cts__d182/i (LVT_INVX20)              0.000   0.032    0.000        0.029 &    4.484 r
clk_ab_go_cts__d182/o (LVT_INVX20)                      0.026                 0.055 &    4.539 f
clk_ab_go_cts__d182n (net)    1    0.113
clk_ab_go_cts__d181/i (LVT_INVX20)              0.000   0.026    0.000        0.026 &    4.565 f
clk_ab_go_cts__d181/o (LVT_INVX20)                      0.013                 0.033 &    4.599 r
clk_ab_go_cts__d181n (net)    1    0.014
clk_ab_go_cts__d181_ctdrc_i1272/i (LVT_INVX20)
                                        0.000    0.013    0.000    0.000 &    4.599 r
clk_ab_go_cts__d181_ctdrc_i1272/o (LVT_INVX20)          0.017                 0.033 &    4.632 f
clk_ab_go_cts__d181_ctdrc_40513 (net)
                              1    0.126
clk_ab_go_cts__d181_ctdrc_i1273/i (LVT_INVX20)
                                        0.000    0.017    0.000    0.041 &    4.673 f
clk_ab_go_cts__d181_ctdrc_i1273/o (LVT_INVX20)          0.011                 0.028 &    4.701 r
clk_ab_go_cts__d181_ctdrc_40514 (net)
                              1    0.015
clk_ab_go_cts__0_0/i (LVT_INVX20)               0.000   0.011    0.000        0.000 &    4.701 r
clk_ab_go_cts__0_0/o (LVT_INVX20)                       0.014                 0.029 &    4.730 f
clk_ab_go_cts__0_0n (net)     1    0.075
clk_ab_go_cts__1_0/i (LVT_INVX20)               0.000   0.014    0.000        0.013 &    4.742 f
clk_ab_go_cts__1_0/o (LVT_INVX20)                       0.030                 0.055 &    4.797 r
clk_ab_go_cts__1_0n (net)     2    0.118
clk_ab_go_cts__d92/i (LVT_INVX20)               0.000   0.030    0.000        0.001 &    4.798 r
clk_ab_go_cts__d92/o (LVT_INVX20)                       0.026                 0.053 &    4.851 f
clk_ab_go_cts__d92n (net)     2    0.130
clk_ab_go_cts__d92n_ctdrc_i1281/i (LVT_INVX4)
                                        0.000    0.026    0.000    0.040 &    4.891 f
clk_ab_go_cts__d92n_ctdrc_i1281/o (LVT_INVX4)           0.012                 0.032 &    4.923 r
clk_ab_go_cts__d92n_ctdrc_40522 (net)
                              1    0.003
pnrt_ctdrc_i1282/i (LVT_INVX4)                  0.000   0.012    0.000        0.000 &    4.923 r
pnrt_ctdrc_i1282/o (LVT_INVX4)                          0.024                 0.048 &    4.972 f
pnrt_ctdrc_40523 (net)        1    0.025
clk_ab_go_cts__d75/i (LVT_INVX20)               0.000   0.024    0.000        0.001 &    4.973 f
clk_ab_go_cts__d75/o (LVT_INVX20)                       0.038                 0.072 &    5.044 r
clk_ab_go_cts__d75n (net)     2    0.168
clk_ab_go_cts__4_0/i (LVT_INVX20)               0.000   0.038    0.000        0.057 &    5.101 r
clk_ab_go_cts__4_0/o (LVT_INVX20)                       0.029                 0.064 &    5.165 f
clk_ab_go_cts__4_0n (net)     2    0.106
    clk_ab_go_cts__d12/i (LVT_INVX20)           0.000    0.029    0.000    0.003 &    5.168 f
clk_ab_go_cts__d12/o (LVT_INVX20)                       0.035                 0.071 &    5.239 r
clk_ab_go_cts__d12n (net)     1    0.128
i_mod1/macroA/clk (macroA_master)
                                        0.000    0.035    0.000    0.039 &    5.277 r
clock reconvergence pessimism                                                 0.000      5.277
inter-clock uncertainty                                                      -0.150      5.127
library setup time                                                           -0.513      4.614
data required time                                                                       4.614
-------------------------------------------------------------------------------------------
data required time                                                                       4.614
data arrival time                                                                       -4.651
-------------------------------------------------------------------------------------------
slack (VIOLATED)                                                                        -0.037
```

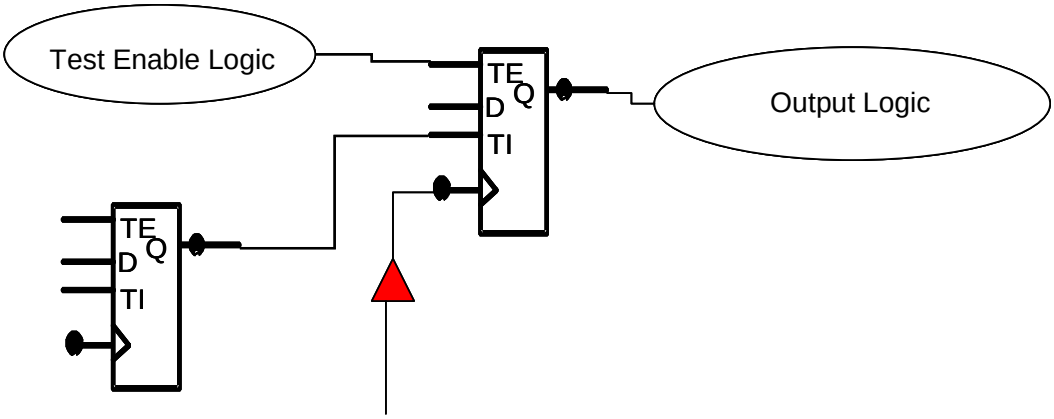
Figure 7: Useful Skew Timing Report.

Consider the timing report shown in Figure 7. Examining the datapath portion of the report (from input “data_in” to macro pin “i_mod1/macroA/macroA_data_in”), we see that all cells are maximally sized and also low Vt. Therefore, no Vt swap or resizing is possible to reduce delay. Also, buffer insertion is unlikely to help since there are no large net delays indicating underbuffering. To eliminate the timing violation, we have no choice but to add useful skew. We choose to add skew by inserting a buffer on the clock pin of the destination register, since the change can be localized to affect only that particular register (in any significant way). Skewing the clock at the source register would impact another designer’s block in this case. It would also require either buffer removal or moving the source register closer to the root of the clock tree, both of which are likely to affect other registers on that clock branch. See Figure 8 for a comparison of source versus destination register useful skew.



If we were to add useful skew at the source register, we would have to bypass or delete the two inverters, as indicated by the dotted line. This could affect timing for the fanout.

Figure 8: Source Versus Destination Register Useful Skew.



Delaying the clock may cause hold violations for the TE and TI pins. It may also cause setup violations at the destinations driven by the “output logic”.

Figure 9: Useful Skew Side Effects.

Useful skew will definitely fix the violation in question but has the potential side effect of creating new violations as well. Delaying the clock to the register may cause hold violations on the test pins and setup violations through the driven logic as illustrated in Figure 9. These should be cleaned up immediately in order for the useful skew insertion to be considered successful and complete. Useful skew is a last resort for fixing setup violations.

B. Buffer Insertion

Although far preferable to useful skew, buffer insertion is still a fairly disruptive operation, especially in congested areas of the design. Since a new cell is added, the placement is locally perturbed, and the place-and-route tool must do an ECO route to reconnect all affected nets. This changes the local noise profile and may introduce new violations. However, buffer insertion is unavoidable in some cases. Consider the timing report segment shown below in Figure 10.

```
 Point                       Fanout    Cap     DTrans    Trans    Delta     Incr       Path
-------------------------------------------------------------------------------------------
 i2050/I (LVT_BUFX16)                          0.000     0.074    0.030    0.104 H    3.801 r
 i2050/o (LVT_BUFX16)                                    0.053             0.207 &    4.008 r
 n_33291 (net)
                               1     0.208
 i976/i (LVT_BUFX16)                           0.000     0.055    0.145    0.065 H    4.073 r
 i976/o (LVT_BUFX16)                                     0.034             0.221 &    4.227 r
 n_15285 (net)
                               1     0.150
 i2017/i (LVT_BUFX16)                          0.000     0.105    0.011    0.164 H    4.391 r
 i2017/o (LVT_BUFX16)                                    0.060             0.220 &    4.612 r
```

Figure 10: Timing Report with Large Crosstalk Delay.

In the report, net “n_33291” induces a large crosstalk delta delay. After checking the layout, we see that net “n_33291” is very long, but its driver is already low Vt and has high drive strength. In this event, we must add a buffer to break the long net.

Other cases that require buffer insertion are as follows. Load splitting is required for high fanout nets or when load cells are spread out. Buffer insertion can also be used to shield particular loads. For example, given a high-fanout net with a small number of timing-critical branches, an inserted buffer can serve as the root of a subtree that drives only non-timing critical branches of the net, thereby reducing the delay for critical branches.

C. Upsizing

Increasing drive strength is slightly less disruptive than buffer insertion, although the placement in the area of the upsized gate may change slightly and ECO routing is needed. When upsizing cells, avoid exceedingly large changes and proceed in a step-by-step manner. Usually, the optimization engine has chosen particular gate sizes for a reason. Sometimes, cells are undersized simply because an associated path does not violate timing at the block level. If a failure only manifests during final, top-level STA, designers may find upsizing very useful and easy to do. Often, a Vt swap can be done simultaneously with an upsize to reduce the power impact of the upsize.

There are certain cases where upsizing is less desirable. For example, gates are poor candidates for upsizing when the instance that drives them has very high fanout. If the driving gate is already

overloaded, upsizing one of its sinks may actually make timing worse. We can easily find and avoid high-fanout gates by referring to the "fanout" column in a timing report.

D. Vt Swap

The simplest and most innocuous of setup fixing changes is Vt swap. This operation costs nothing in terms of routability since cell footprints are typically the same across all Vt classes. In fact, it is not even necessary to re-extract after performing Vt swap. Because Vt swap is not invasive, it is the most desirable operation for fixing setup violations and should be used whenever possible. Consider the timing report segment in Figure 11. Clearly, there are several candidates for Vt swap since the path contains multiple high Vt cells. Still, when selecting a cell to swap, we try to choose one with an anomalous delay. In this case, we select "G1002" and will change it to standard Vt (SVT_NAND4X1).

```
 Point                      Fanout   Cap    DTrans   Trans    Delta     Incr       Path
-------------------------------------------------------------------------------------
 G1001/i (SVT_BUFX2)                        0.000    0.074    0.030    0.104 H    3.801 r
 G1001/o (SVT_BUFX2)                                 0.053             0.207 &    4.008 r
 n_11360(net)
                            5    0.159
 G1002/i1 (HVT_NAND4X1)                     0.000    0.105    0.040    0.065 H    4.073 r
 G1002/o  (HVT_NAND4X1)                              0.034             0.521 &    4.527 r
 n_11361 (net)
                            1    0.150
 G1003/i (HVT_NOR2X1)                       0.000    0.097    0.011    0.164 H    4.691 r
 G1003/o (HVT_NOR2X1)                                0.060             0.220 &    4.912 r
```

Figure 11: Vt Swap Timing Report.

Note also that, in some standard cell libraries, engineers can also exploit same-footprint cells with different drive strengths. For example, INVX2, INVX3 and INVX4 might have the same footprint. In that case, upsizing INVX2 to INVX4 is effectively the "same cost" as a Vt swap operation and may be a better way to fix timing in last-stage STA repair. However, not all libraries have this nice feature.

VIII. FIXING HOLD VIOLATIONS

Hold fixing is done concurrently with setup fixing during manual repair. In fact, it should be completed in the earlier manual repair iterations to the greatest extent possible since it normally requires buffer insertion. Fortunately, few hold violations typically manifest during the final stages of manual repair (although this may change for newer process nodes 40nm and below). The guiding principle during manual hold repair is isolation. We really don't want to touch anything except the failing endpoint, as far as this is possible. Operations used to fix hold violations include the following:

1) Buffer insertion.
2) Changing flip-flops to high Vt, instead of standard or low Vt.

We can formulate some general guidelines to help with isolation as well.

1) Whenever possible, insert hold buffers just before the D pin of the failing register.
2) When paths have widely mixed Vt classes, insert hold buffers in the high Vt section of the path. We should assume that the optimization tool has made certain gates low Vt for a reason.
3) When possible, avoid inserting hold buffers prior to very high fanout gates. Delaying such a gate will slow down all paths through that gate and its fanouts, some of which may be critical.

Again, it is best to localize the ripple effects of changes. Figure 12 shows a sample timing report for a hold time violation. Looking carefully at the report, we see that the violation was probably caused by

a useful skew buffer insertion (i_blkA/BUF_SKEW_27). Already, one hold buffer has been added in an attempt to fix the problem (i_blkA/BUF_HOLD_3). We can add another buffer on the “d” pin of the destination flip-flop to eliminate the remaining negative slack.

```
Startpoint: i_blkA/enable_reg_19_
            (rising edge-triggered flip-flop clocked by clk)
Endpoint: i_blkA/write_reg
            (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: min
Min Data Paths Derating Factor  : 0.900(cell)  0.900(net)
Min Clock Paths Derating Factor : 0.950(cell)  0.950(net)
Max Clock Paths Derating Factor : 1.100(cell)  1.100(net)

Point                      Fanout   Cap    DTrans   Trans    Delta     Incr       Path
-------------------------------------------------------------------------------------
clock clk (rise edge)                                                  0.000      0.000
clock source latency                                                   0.000      0.000
clk (in)                                             0.005             0.002 &    0.002 r
clk (net)                    1    0.004
Ubuf_clk/i (LVT_CKBUFX8)                    0.000    0.005    0.000    0.000 &    0.002 r
Ubuf_clk/o (LVT_CKBUFX8)                             0.013             0.038 &    0.040 r
clk0 (net)                   2    0.081
clk_go_cts__d6067/i (LVT_INVX20)            0.000    0.013    0.000    0.022 &    0.062 r
clk_go_cts__d6067/o (LVT_INVX20)                     0.008             0.012 &    0.074 f
clk_go_cts__d6067n (net)     2    0.046
clk_go_cts__d6067n_ctdrc_i1113/i (LVT_INVX10)
                                            0.000    0.008    0.000    0.007 &    0.081 f
clk_go_cts__d6067n_ctdrc_i1113/o (LVT_INVX10)        0.009             0.017 &    0.098 r
clk_go_cts__d6067n_ctdrc_40354 (net)
                             1    0.027
clk_go_cts__d6066_ctdrc_i1108/i (LVT_INVX20)
                                            0.000    0.009    0.000    0.000 &    0.098 r
clk_go_cts__d6066_ctdrc_i1108/o (LVT_INVX20)         0.006             0.011 &    0.109 f
clk_go_cts__d6066_ctdrc_40349 (net)
                             1    0.057
clk_go_cts__d6066_ctdrc_i1109/i (LVT_INVX20)
                                            0.000    0.006    0.000    0.009 &    0.118 f
clk_go_cts__d6066_ctdrc_i1109/o (LVT_INVX20)         0.008             0.014 &    0.133 r
clk_go_cts__d6066_ctdrc_40350 (net)
                             2    0.105
clk_go_cts__d6065/i (LVT_INVX20)            0.000    0.008    0.000    0.033 &    0.166 r
clk_go_cts__d6065/o (LVT_INVX20)                     0.006             0.010 &    0.176 f
clk_go_cts__d6065n (net)     1    0.062
clk_go_cts__d6064/i (LVT_INVX20)            0.000    0.006    0.000    0.012 &    0.188 f
clk_go_cts__d6064/o (LVT_INVX20)                     0.008             0.014 &    0.202 r
clk_go_cts__d6064n (net)     2    0.132
clk_go_cts__2_0/i (LVT_INVX20)              0.000    0.008    0.000    0.050 &    0.252 r
clk_go_cts__2_0/o (LVT_INVX20)                       0.006             0.010 &    0.262 f
clk_go_cts__2_0n (net)       1    0.049
clk_go_cts__d5949/i (LVT_INVX20)            0.000    0.006    0.000    0.008 &    0.270 f
clk_go_cts__d5949/o (LVT_INVX20)                     0.005             0.011 &    0.281 r
clk_go_cts__d5949n (net)     1    0.025
clk_go_cts__d5948/i (LVT_INVX20)            0.000    0.005    0.000    0.000 &    0.281 r
clk_go_cts__d5948/o (LVT_INVX20)                     0.005             0.009 &    0.290 f
clk_go_cts__d5948n (net)     1    0.064
clk_go_cts__d5947/i (LVT_INVX20)            0.000    0.005    0.000    0.012 &    0.302 f
clk_go_cts__d5947/o (LVT_INVX20)                     0.010             0.017 &    0.320 r
clk_go_cts__d5947n (net)     2    0.142
clk_go_cts__d5908/i (LVT_INVX20)            0.000    0.010    0.000    0.002 &    0.322 r
clk_go_cts__d5908/o (LVT_INVX20)                     0.005             0.010 &    0.332 f
clk_go_cts__d5908n (net)     2    0.028
clk_go_cts__d5908_ctdrc_i1160/i (LVT_INVX10)
                                            0.000    0.005    0.000    0.000 &    0.332 f
clk_go_cts__d5908_ctdrc_i1160/o (LVT_INVX10)         0.007             0.015 &    0.347 r
clk_go_cts__d5908_ctdrc_40401 (net)
                             1    0.025
clk_go_cts__d5908_ctdrc_i1161/i (LVT_INVX20)
                                            0.000    0.007    0.000    0.000 &    0.347 r
```

```
clk_go_cts__d5908_ctdrc_i1161/o (LVT_INVX20)        0.006            0.010 &    0.357 f
clk_go_cts__d5908_ctdrc_40402 (net)
                          2    0.089
clk_go_cts__d5882/i (LVT_INVX20)           0.000    0.006    0.000    0.016 &    0.373 f
clk_go_cts__d5882/o (LVT_INVX20)                    0.007            0.014 &    0.387 r
clk_go_cts__d5882n (net)     2    0.041
clk_go_cts__6_7/i (LVT_INVX20)             0.000    0.007    0.000    0.000 &    0.387 r
clk_go_cts__6_7/o (LVT_INVX20)                      0.009            0.016 &    0.404 f
clk_go_cts__6_7n (net)       4    0.207
clk_go_cts__7_36/i (LVT_INVX20)            0.000    0.009    0.000    0.007 &    0.411 f
clk_go_cts__7_36/o (LVT_INVX20)                     0.031            0.042 &    0.453 r
clk_go_cts__7_36n (net)      9    0.327
clk_go_cts__8_264/i (LVT_INVX20)           0.000    0.031    0.000    0.044 &    0.497 r
clk_go_cts__8_264/o (LVT_INVX20)                    0.028            0.045 &    0.541 f
clk_go_cts__8_264n (net)    12    0.252
clk_go_cts__d2382/i (LVT_INVX20)           0.000    0.028    0.000    0.000 &    0.542 f
clk_go_cts__d2382/o (LVT_INVX20)                    0.009            0.010 &    0.552 r
clk_go_cts__d2382n (net)     1    0.005
i_blkA/blkA_clk_gate_en/phi (LVT_CKENOAX8)
                                           0.000    0.009    0.000    0.000 &    0.552 r
i_blkA/blkA_clk_gate_en/o (LVT_CKENOAX8)
                                                    0.020            0.056 &    0.608 r
i_blkA/blkA_clk_gate_en_0 (net)
                          28    0.102
i_blkA/enable_reg_19_/phi (HVT_SDFFX1)
                                           0.000    0.020    0.000    0.001 &    0.610 r
i_blkA/enable_reg_19_/q (HVT_SDFFX1)
                                                    0.041            0.106 &    0.716 r
i_blkA/enable_19 (net)
                          4    0.014
i_blkA/BUF_HOLD_3/i (LVT_DLY150X3)         0.000    0.041    0.000    0.000 &    0.716 r
i_blkA/BUF_HOLD_3/o (LVT_DLY150X3)                  0.010            0.103 &    0.819 r
i_blkA/n_buf_hold_3 (net)
                          1    0.001
i_blkA/write_reg/d (LVT_SDFFX2)
                                           0.000    0.010    0.000    0.000 &    0.819 r
data arrival time                                                                 0.819

clock clk (rise edge)                                                0.000      0.000
clock source latency                                                 0.000      0.000
clk (in)                                            0.027            0.003 &    0.003 r
clk (net)                 1    0.004
Ubuf_clk/i (LVT_CKBUFX8)                   0.000    0.027    0.000    0.000 &    0.003 r
Ubuf_clk/o (LVT_CKBUFX8)                            0.016            0.056 &    0.060 r
clk0 (net)                2    0.081
clk_go_cts__d6067/i (LVT_INVX20)           0.000    0.023    0.000    0.023 &    0.082 r
clk_go_cts__d6067/o (LVT_INVX20)                    0.011            0.015 &    0.098 f
clk_go_cts__d6067n (net)     2    0.046
clk_go_cts__d6067n_ctdrc_i1113/i (LVT_INVX10)
                                           0.000    0.012    0.000    0.007 &    0.105 f
clk_go_cts__d6067n_ctdrc_i1113/o (LVT_INVX10)       0.010            0.021 &    0.126 r
clk_go_cts__d6067n_ctdrc_40354 (net)
                          1    0.027
clk_go_cts__d6066_ctdrc_i1108/i (LVT_INVX20)
                                           0.000    0.010    0.000    0.000 &    0.126 r
clk_go_cts__d6066_ctdrc_i1108/o (LVT_INVX20)        0.007            0.012 &    0.138 f
clk_go_cts__d6066_ctdrc_40349 (net)
                          1    0.057
clk_go_cts__d6066_ctdrc_i1109/i (LVT_INVX20)
                                           0.000    0.010    0.000    0.010 &    0.148 f
clk_go_cts__d6066_ctdrc_i1109/o (LVT_INVX20)        0.010            0.018 &    0.166 r
clk_go_cts__d6066_ctdrc_40350 (net)
                          2    0.105
clk_go_cts__d6065/i (LVT_INVX20)           0.000    0.025    0.000    0.035 &    0.200 r
clk_go_cts__d6065/o (LVT_INVX20)                    0.013            0.017 &    0.218 f
clk_go_cts__d6065n (net)     1    0.062
clk_go_cts__d6064/i (LVT_INVX20)           0.000    0.016    0.000    0.013 &    0.231 f
clk_go_cts__d6064/o (LVT_INVX20)                    0.014            0.022 &    0.252 r
clk_go_cts__d6064n (net)     2    0.132
clk_go_cts__2_0/i (LVT_INVX20)             0.000    0.038    0.000    0.052 &    0.305 r
clk_go_cts__2_0/o (LVT_INVX20)                      0.016            0.018 &    0.322 f
```

```
clk_go_cts__2_0n (net)       1    0.049
clk_go_cts__d5949/i (LVT_INVX20)           0.000    0.017    0.000    0.008 &    0.331 f
clk_go_cts__d5949/o (LVT_INVX20)                    0.009             0.016 &    0.346 r
clk_go_cts__d5949n (net)     1    0.025
clk_go_cts__d5948/i (LVT_INVX20)           0.000    0.009    0.000    0.000 &    0.346 r
clk_go_cts__d5948/o (LVT_INVX20)                    0.006             0.011 &    0.358 f
clk_go_cts__d5948n (net)     1    0.064
clk_go_cts__d5947/i (LVT_INVX20)           0.000    0.011    0.000    0.013 &    0.371 f
clk_go_cts__d5947/o (LVT_INVX20)                    0.013             0.023 &    0.394 r
clk_go_cts__d5947n (net)     2    0.142
clk_go_cts__d5908/i (LVT_INVX20)           0.000    0.014    0.000    0.002 &    0.396 r
clk_go_cts__d5908/o (LVT_INVX20)                    0.007             0.011 &    0.408 f
clk_go_cts__d5908n (net)     2    0.028
clk_go_cts__d5908_ctdrc_i1160/i (LVT_INVX10)
                                           0.000    0.007    0.000    0.001 &    0.408 f
clk_go_cts__d5908_ctdrc_i1160/o (LVT_INVX10)        0.008             0.017 &    0.425 r
clk_go_cts__d5908_ctdrc_40401 (net)
                             1    0.025
clk_go_cts__d5908_ctdrc_i1161/i (LVT_INVX20)
                                           0.000    0.008    0.000    0.000 &    0.425 r
clk_go_cts__d5908_ctdrc_i1161/o (LVT_INVX20)        0.006             0.011 &    0.436 f
clk_go_cts__d5908_ctdrc_40402 (net)
                             2    0.089
clk_go_cts__d5882/i (LVT_INVX20)           0.000    0.015    0.000    0.016 &    0.453 f
clk_go_cts__d5882/o (LVT_INVX20)                    0.010             0.019 &    0.472 r
clk_go_cts__d5882n (net)     2    0.041
clk_go_cts__6_7/i (LVT_INVX20)             0.000    0.010    0.000    0.000 &    0.472 r
clk_go_cts__6_7/o (LVT_INVX20)                      0.011             0.020 &    0.492 f
clk_go_cts__6_7n (net)       4    0.207
clk_go_cts__7_36/i (LVT_INVX20)            0.000    0.016    0.000    0.007 &    0.500 f
clk_go_cts__7_36/o (LVT_INVX20)                     0.033             0.051 &    0.551 r
clk_go_cts__7_36n (net)      9    0.327
clk_go_cts__8_258/i (LVT_INVX20)           0.000    0.048    0.000    0.032 &    0.583 r
clk_go_cts__8_258/o (LVT_INVX20)                    0.037             0.060 &    0.643 f
clk_go_cts__8_258n (net)    12    0.273
clk_go_cts__9_176/i (LVT_INVX20)           0.000    0.039    0.000    0.018 &    0.661 f
clk_go_cts__9_176/o (LVT_INVX20)                    0.036             0.067 &    0.728 r
clk_go_cts__9_176n (net)    46    0.217
i_blkA/BUF_SKEW_27/i (LVT_DLY250X3)        0.000    0.036    0.000    0.001 &    0.729 r
i_blkA/BUF_SKEW_27/o (LVT_DLY250X3)                 0.013             0.210 &    0.938 r
i_blkA/eco_insert_buff1387 (net)
                             1    0.004
i_blkA/write_reg/phi (LVT_SDFFX2)
                                           0.000    0.013    0.000    0.000 &    0.938 r
clock reconvergence pessimism                                        -0.098      0.840
clock uncertainty                                                     0.040      0.880
library hold time                                                    -0.034      0.846
data required time                                                                0.846
-------------------------------------------------------------------------------------
data required time                                                                0.846
data arrival time                                                                -0.819
-------------------------------------------------------------------------------------
slack (VIOLATED)                                                                 -0.027
```

Figure 12: Hold Violation Timing Report.

Finally, some failures are easily fixed by changing registers from low or standard Vt to high Vt. Optimization tools often do not touch registers; however, flip-flops are usually not as sacred as the tools regard them. If the functional path through the register is not critical, changing to high Vt has no negative impact on timing.

IX.CONCLUSION

From the above discussion, we conclude that late-stage timing repair is still more of an art than a science. Signoff timing closure virtually always includes manual repair, and it is still critical for designers to learn how to perform this task quickly and effectively. In addition, having a solid methodology to manage/accelerate convergence is imperative. Methodology also helps to integrate new designers into the team and to teach new college graduates skills for success. In the end, there is still no substitute for experience.